Query Reliability Mechanism

Table of Contents

- I. Overall Architecture Overview
  - 1.1 Design Philosophy: Query as a State Machine
  - 1.2 Dual-Layer Entry Points
  - 1.3 Design Intent of Loop State (State)
- II. Context Compression and Overflow Recovery
  - 2.1 Proactive Compression Chain (Proactive → Autocompact)
  - 2.2 Reactive Compression Chain (Reactive)
  - 2.3 Blocking Limit Check
- III. API Retry and Model Fault Tolerance
  - 3.0 Where the Two-Layer Error Handling Boundary Lies
  - 3.1 `withRetry` Wrapper (`services/api/withRetry.ts`)
  - 3.2 Model Fallback
- IV. Streaming Fault Tolerance
  - 4.1 Streaming Tool Executor
  - 4.2 Orphan Message Handling (Tombstoning)
  - 4.3 Tool Result Budget (Tool Result Size Limit)
- V. Max Output Tokens Recovery
  - 5.1 Level One: Token Limit Escalation
  - 5.2 Level Two: Multi-Round Recovery (`MAX_OUTPUT_TOKENS_RECOVERY_LIMIT = 3`)
  - 5.3 Level Three: Expose Error
- VI. Token Budget Mechanism
  - 6.1 Design Intent of Diminishing Returns Detection
  - 6.2 `lastGlobalTurnTokens` and Cross-Turn Accumulation Tracking
- VII. Stop Hooks Mechanism
  - 7.1 Execution Flow
  - 7.2 Blocking Handling
  - 7.3 Prevent Continuation
  - 7.4 Teammate Hook Chain
- VIII. User Interruption Handling
  - 8.1 Streaming Interruption (User Ctrl+C)
  - 8.2 Tool Execution Interruption
  - 8.3 Submit Interrupt
- IX. Async Prefetch and Parallelization
  - 9.1 Memory Prefetch
  - 9.2 Skill Discovery Prefetch
  - 9.3 Tool Use Summary
- X. Tool Use Context Refresh
- XI. Error Classification and Handling

- 11.1 Error Type Mapping
- 11.2 Safety Guarantees for Error Exposure
- XII. Feature Flags Summary
  - 12.1 `querySource` as a Typed Security Policy Context
- XIII. Design Philosophy Summary
  - 13.0 The "Three-Layer Funnel" Philosophy of Error Handling
  - 13.1 Defensive Streaming Handling
  - 13.2 Withheld Mode
  - 13.3 Immutable State Transitions
  - 13.4 Multi-Layer Degradation Strategy
  - 13.5 Dependency Injection: Intentional Design for Testability
  - 13.6 529 Overload "Amplification Attack" Protection
  - 13.7 Background Task "Avalanche Prevention" Isolation
  - 13.8 QueryCheckpoint Dual-Layer Performance Tracking System
  - 13.9 "Implicit Dependency" Awareness in Tool Execution
  - 13.10 Analytics as a First-Class Citizen of Architecture
  - 13.11 `using` Declarations and Deterministic Resource Management
  - 13.12 Death Spiral Protection
- XIV. Key Data Flow Diagram
- XV. Related Files Quick Reference

I. Overall Architecture Overview

`query.ts` is Claude Code's **main query engine**, responsible for driving the complete dialogue loop between the user and the LLM. It is implemented as an `async generator` supporting streaming output, and inserts extensive reliability mechanisms between each API call to handle runtime failures such as context overflow, rate limiting, model overload, and user interruptions.

1.1 Design Philosophy: Query as a State Machine

The key to understanding query.ts is not memorizing every recovery path, but understanding **why** it is designed as an infinite-loop **state machine** rather than a sequential function chain. Three core principles underpin this choice:

Principle One: LLM dialogue is inherently non-deterministic. A single API call may fail due to context overflow (requiring compression), wait due to rate limiting (requiring retry), need degradation due to model overload (requiring model switch), or be interrupted because the user pressed Ctrl+C (requiring cleanup). These failures cannot all be predicted before the call; they must be handled one by one at runtime. Wrapping the entire flow in a `while(true)` loop gives every failure a unified "continue iteration, retry" exit, eliminating the need for separate retry branches for each error type.

Principle Two: Streaming output demands real-time intervention. Claude Code is not a batch processing system that "waits for the model to fully respond before processing." It begins **executing tools during the streaming output process**. This creates a fundamental tension: once `yield` starts streaming data, the control flow has already spread to the caller (SDK/REPL). The `async generator` design solves this — `yield` hands control back to the query loop while streaming continues, allowing the loop to process the next tool result while

the caller continues consuming the already-yielded data. This is a form of "cooperative concurrency": the generator and caller each advance their own progress, coordinated through `yield`.

Principle Three: Immutable state transitions prevent "ghost bugs". There are 9 `continue` sites in the query loop, each of which must **update all** cross-iteration state. If mutable field assignments were used (e.g., `state.field = newValue`), one `continue` site forgetting to reset a field would cause state leakage, leading to hard-to-trace ghost bugs. **Replacing the entire `State` object with a new one** means each `continue` site either explicitly sets every field (via spread) or inherits the old value (via `...(previous)`) — a missing field equals a missing line of code, immediately visible in code review.

Core file structure:

```
query.ts                          # Main query loop (entry + core while(true) loop)
├── query/stopHooks.ts            # Stop Hooks executor (hook chain at turn end)
├── query/tokenBudget.ts          # Token budget tracking and early stop decisions
├── services/api/withRetry.ts     # API call retry wrapper
└── [other supporting modules]
    autoCompact / reactiveCompact / contextCollapse / microcompact / snipCompact
    StreamingToolExecutor / runTools
```

1.2 Dual-Layer Entry Points

query.ts intentionally separates the public API (`query`) and the internal loop body (`queryLoop`) into two functions:

```
// Public entry: responsible for cleaning up consumedCommandUuids
export async function* query(params: QueryParams): AsyncGenerator<...> {
  const consumedCommandUuids: string[] = []
  const terminal = yield* queryLoop(params, consumedCommandUuids)
  for (const uuid of consumedCommandUuids) {
    notifyCommandLifecycle(uuid, 'completed')
  }
  return terminal
}

// Internal loop body: core carrier of all reliability mechanisms
async function* queryLoop(params, consumedCommandUuids): AsyncGenerator<...> {
  // while(true) infinite loop, continues via continue jumps
  // Exits loop via return { reason: 'xxx' }
}
```

The `query()` layer handles "session lifecycle management": when `queryLoop` returns normally (rather than throwing or returning), it notifies all consumed commands as completed. It is the "cleanup hook" for the entire generator chain. `queryLoop()` carries all runtime reliability logic — this is the core reason for separating them. Merging them would entangle reliability logic with session cleanup, increasing testing and modification complexity.

On the use of `yield*`: `yield*` (delegate generation) is used rather than plain `yield` because when an internal generator of `queryLoop` throws, the exception propagates directly through `query()` to the caller without needing an explicit try/catch. This provides the same exception-transparent propagation semantics as regular functions, allowing SDK consumers (e.g., cowork/desktop) to catch all exceptions with a unified `try/catch`.

> **Note**
>
> - If an exception is thrown inside queryLoop, it appears directly in the caller's catch — the caller has no idea the `query` function even existed
> - This is the same logic as `return` passing values: yield* lets exceptions also go through "direct穿透" rather than being forwarded through an intermediary

1.3 Design Intent of Loop State (State)

The entire reliability mechanism is built on **cross-iteration state persistence**. The `State` type aggregates all mutable state that needs to survive across loop iterations:

```
type State = {
  messages: Message[]                     // Conversation history
  toolUseContext: ToolUseContext          // Tool context
  autoCompactTracking: AutoCompactTrackingState | undefined  // Auto-compaction state
  maxOutputTokensRecoveryCount: number  // max_output_tokens recovery counter
  hasAttemptedReactiveCompact: boolean  // Whether reactive compaction has been attempted
  maxOutputTokensOverride: number | undefined  // Output token limit override
  pendingToolUseSummary: Promise<...> | undefined  // Tool use summary (async)
  stopHookActive: boolean | undefined   // Whether a Stop Hook is active
  turnCount: number                     // Current turn count
  transition: Continue | undefined      // Reason for previous iteration's continuation
(debugging/testing)
}
```

Each `continue` replaces the entire state object via `state = { ...nextState }` — an **immutable state transition** pattern that avoids scattered local variable assignments.

II. Context Compression and Overflow Recovery

> **Note**
>
> **See the "Context Management" documentation for details**

This is the most complex reliability subsystem in query.ts, divided into **proactive compression** and **reactive compression** paths.

2.1 Proactive Compression Chain (Proactive → Autocompact)

Before the API call, the following compression steps are executed in order:

2.1.1 History Snip (`HISTORY_SNIP` Feature Flag)

```
// restored-src/src/services/compact/snipCompact.ts
if (feature('HISTORY_SNIP')) {
  const snipResult = snipModule!.snipCompactIfNeeded(messagesForQuery)
  messagesForQuery = snipResult.messages
  snipTokensFreed = snipResult.tokensFreed  // Passed back to autocompact threshold check
}
```

- Goal: Remove "old enough" and "no longer referenced" historical messages from the context window
- Key point: The number of tokens freed (`snipTokensFreed`) is passed to autocompact so its threshold check reflects what snip has already removed

> **Note**
>
> This is the truncation feature — a rather blunt context operation. In the current version of CC, **this feature is completely silently disabled**.

2.1.2 Microcompact

```
const microcompactResult = await deps.microcompact(messagesForQuery, toolUseContext,
querySource)
messagesForQuery = microcompactResult.messages
```

- **Cached Microcompact** (`CACHED_MICROCOMPACT` feature): Caching optimization for edit-type tool results, using the API-returned `cache_deleted_input_tokens` field to precisely calculate deletion amounts
- Defers yielding boundary messages until after the API response, to use the actual `cache_deleted_input_tokens` reported by the API

> **Note**
>
> **This is the cache_edit capability. See the "Context Management" documentation for details**

2.1.3 Context Collapse (`CONTEXT_COLLAPSE` Feature)

```
if (feature('CONTEXT_COLLAPSE') && contextCollapse) {
  const collapseResult = await contextCollapse.applyCollapsesIfNeeded(...)
  messagesForQuery = collapseResult.messages
}
```

- A commit-log-based collapse mechanism: collapse results are persisted in collapse storage rather than directly modifying the REPL array
- This makes collapses persist across multiple turns: each `projectView()` replays the commit log on a fresh entry

2.1.4 Autocompact

```
const { compactionResult, consecutiveFailures } = await deps.autocompact(...)
```

- Summarizes and compresses early messages when token count exceeds the threshold
- The `consecutiveFailures` field tracks consecutive failure count, used for **circuit breaker** logic
- On successful compression, reset the `tracking` state:

```
tracking = {
  compacted: true,
  turnId: deps.uuid(),
  turnCounter: 0,         // Reset turn counter
```

```
    consecutiveFailures: 0, // Reset consecutive failure counter
  }
```

Analytics instrumentation: After successful compression, records detailed token statistics (pre/post compression token counts, cache hit/miss, etc.) for cost analysis.

task_budget cross-compression-boundary tracking: Serves as an upper-bound guardrail for resource consumption, preventing the Agent from bypassing sub-agent derivation limits either intentionally or unintentionally after context compression causes memory loss, ensuring the total number of sub-agents across the entire session lifecycle (spanning any number of compressions) never exceeds the configured safety threshold.

2.2 Reactive Compression Chain (Reactive)

When proactive compression fails to prevent a **413 Prompt Too Long** error, the reactive path is triggered:

2.2.1 Withheld Mechanism (Error Suspension)

In the streaming loop, recoverable errors are **withheld** without immediately exposing them to the caller:

```
let withheld = false
if (contextCollapse?.isWithheldPromptTooLong(message, ...)) withheld = true
if (reactiveCompact?.isWithheldPromptTooLong(message)) withheld = true
if (mediaRecoveryEnabled && reactiveCompact?.isWithheldMediaSizeError(message)) withheld =
true
if (isWithheldMaxOutputTokens(message)) withheld = true
if (!withheld) { yield yieldMessage }  // Only yield when not withheld
```

> **Note**
>
> **Design intent**: If errors are exposed too early, SDK consumers (e.g., cowork/desktop) terminate the session upon seeing an `error` field, leaving recovery logic orphaned.

2.2.2 Two-Phase Recovery

Phase One: Context Collapse Queue Drain (triggered only when the previous round was not `collapse_drain_retry`)

```
if (feature('CONTEXT_COLLAPSE') && state.transition?.reason !== 'collapse_drain_retry') {
  const drained = contextCollapse.recoverFromOverflow(messagesForQuery, querySource)
  if (drained.committed > 0) {
    state = { ...state, transition: { reason: 'collapse_drain_retry', committed:
drained.committed } }
    continue  // Retry API call
  }
}
```

> **Note**
>
> Core idea: instead of compressing the entire conversation, **replace message chunks already marked as "collapsible" (e.g., intermediate results already referenced by the model and no longer needing full preservation) with shorter placeholders**.

Phase Two: Reactive Compact (if Phase One fails to resolve)

```
const compacted = await reactiveCompact.tryReactiveCompact({
  hasAttempted: hasAttemptedReactiveCompact,
  querySource,
  messages: messagesForQuery,
  ...
})
if (compacted) {
  state = { ...state, transition: { reason: 'reactive_compact_retry' } }
  continue
}
```

Deadlock prevention: If both recovery attempts fail, **do not fall through to stop hooks** — because the model never produced a valid response, stop hooks would cause a death spiral: error → hook blocking → retry → error → …

2.3 Blocking Limit Check

When **proactive compression is disabled**, a hard blocking limit check is performed before the API call:

```
if (!compactionResult && querySource !== 'compact' && querySource !== 'session_memory'
    && !(reactiveCompact?.isReactiveCompactEnabled() && isAutoCompactEnabled())
    && !collapseOwnsIt) {
  const { isAtBlockingLimit } = calculateTokenWarningState(
    tokenCountWithEstimation(messagesForQuery) - snipTokensFreed,
    toolUseContext.options.mainLoopModel,
  )
  if (isAtBlockingLimit) {
    yield createAssistantAPIErrorMessage({ content: PROMPT_TOO_LONG_ERROR_MESSAGE, error:
'invalid_request' })
    return { reason: 'blocking_limit' }
  }
}
```

- Skip conditions: compression just completed (verified below threshold), is a compact/session_memory query, reactive compact or context collapse is enabled

---

III. API Retry and Model Fault Tolerance

3.0 Where the Two-Layer Error Handling Boundary Lies

query.ts actually has two distinct retry layers, and **the boundary between them needs careful attention**:

Layer One (`withRetry.ts`): Handles **transport layer and gateway layer** errors — 429 rate limits, 529 overloads, 401/403 authentication, 408 timeouts, ECONNRESET/EPIPE connection issues. The common trait of these errors: they have **nothing to do with the content of the request itself**, purely infrastructure-level issues. Retrying the same content (identical messages) usually succeeds.
Layer Two (query.ts loop): Handles **semantic layer** errors — 413 PTL, max_output_tokens, model degradation. The common trait of these errors: they are **directly related to the content of the request**, and can

only be resolved by modifying the request content (compressing context, switching models, injecting recovery messages).

The rationale for this layering: **semantic layer modifications must abandon the current request (cannot simply retry)**, while transport layer errors **can retry the same request**.

3.1 `withRetry` Wrapper (`services/api/withRetry.ts`)

```
async function* withRetry<T>(
  getClient: () => Promise<Anthropic>,
  operation: (client, attempt, context) => Promise<T>,
  options: RetryOptions,
): AsyncGenerator<SystemAPIErrorMessage, T>
```

3.1.1 Retry Strategy

Error Type	Handling Strategy
408 Request Timeout	Retry (exponential backoff + jitter)
409 Lock Timeout	Retry
429 Rate Limit	Retry (non-subscribers only; enterprise users can retry due to PAYG)
529 Overloaded	Retry (`MAX_529_RETRIES=3` , triggers Model Fallback)
5xx Server Error	Retry
APIConnectionError (ECONNRESET/EPIPE)	Retry and disable keep-alive
401/403 Authentication Error	Clear cache + refresh token, then retry
max_tokens exceeds context limit	Dynamically lower `max_tokens` and retry

3.1.2 Backoff Formula

```
const baseDelay = Math.min(BASE_DELAY_MS * 2^(attempt-1), maxDelayMs)  // maxDelayMs=32000
const jitter = Math.random() * 0.25 * baseDelay
return baseDelay + jitter
```

> **Note**
>
> Exponential Backoff with Jitter

3.1.2.1 Priority of `Retry-After` Header

Notably, the backoff strategy **prioritizes the server-returned** `Retry-After` header (if present):

```
if (retryAfterHeader) {
  return parseInt(retryAfterHeader, 10) * 1000  // Directly use server's recommendation
}
```

> **Note**
>
> The server knows its own current load situation better than the client

3.1.3 Persistent Retry Mode (`UNATTENDED_RETRY`)

Suitable for long-running unattended sessions (e.g., background agent / CI pipeline):

```
const PERSISTENT_MAX_BACKOFF_MS = 5 * 60 * 1000  // 5 minute cap
const PERSISTENT_RESET_CAP_MS = 6 * 60 * 60 * 1000  // 6 hour cap
const HEARTBEAT_INTERVAL_MS = 30_000  // Yield every 30s as anti-idle signal
```

- Long sleeps are broken into 30-second chunks, with `AbortSignal` checked after each chunk
- Periodically yields `{type:'system', subtype:'api_retry'}` to prevent the host from marking the session as idle

> **Note**
>
> The defining characteristic of these tasks: **they'd rather wait for the API to recover than expect failure**

3.1.4 Fast Mode Degradation

Handling of 429/529 in Fast Mode:

```
if (retryAfterMs < SHORT_RETRY_THRESHOLD_MS) {  // < 20s
  await sleep(retryAfterMs); continue  // Stay in fast mode, retry (better to wait short
delays; degradation invalidates cache, causing significant loss)
} else {
  triggerFastModeCooldown(Date.now() + cooldownMs)  // Degrade to standard speed
  retryContext.fastMode = false; continue
}
```

> **Note**
>
> - Fast Mode is a reasoning acceleration feature in Claude Code. Its core principle is using the API's **Prompt Caching** to reduce token consumption per request, thereby lowering latency and cost
> - 429 = you're rate-limited, wait a bit; 529 = server is about to collapse, don't add to the load

3.1.5 Foreground/Background Distinction

Only the following `querySource` values retry on 529 (user waiting in foreground):

```
const FOREGROUND_529_RETRY_SOURCES = new Set([
  'repl_main_thread', 'sdk', 'agent:*', 'compact', 'hook_agent',
  'hook_prompt', 'verification_agent', 'side_question', 'auto_mode', ...
])
```

Other background sources (e.g., summary generation, title generation) encountering 529 **fail immediately**, avoiding amplification of gateway pressure during capacity overload.

> **Note**
>
> **Foreground sources (user-aware, retriable) — only these 4 categories:**
>
> 1. REPL main loop — user typing in terminal, waiting to see output

2. Agent execution — includes custom agents, built-in agents, agents invoked in hooks
3. Compact — context compression, imperceptible to user but task itself is synchronously completed
4. Synchronous safety judgments — auto_mode classifier, side_question, verification_agent
   Core principle: there is a real person sitting in front of the screen waiting for this result.

Background sources (user-unaware, non-retriable):

- Title/name generation: session titles, rename new names, Teleport titles
- Memory system: memory extraction, relevance search, cross-session search, session_memory
- Agent lifecycle: agent creation, agent summary, agent recovery
- Hooks: prompt_suggestion, speculation, skill_improvement
- External integrations: Chrome MCP, web search, web_fetch, mcp_datetime_parse
- Auxiliary analytics: insights, auto_dream, away_summary, magic_docs, feedback
- Tool-related: tool_use_summary_generation, bash_extract_prefix, permission_explainer, model_validation
  Core principle: the user has no idea and doesn't care whether these requests succeed or fail. Failed silently if they don't work; no need to add chaos when the API is overloaded.

3.2 Model Fallback

When 3 consecutive 529 errors occur, model fallback is triggered:

```
// in withRetry.ts
if (consecutive529Errors >= MAX_529_RETRIES) {
  if (options.fallbackModel) {
    throw new FallbackTriggeredError(options.model, options.fallbackModel)
  }
}

// Caught in query.ts
if (innerError instanceof FallbackTriggeredError && fallbackModel) {
  currentModel = fallbackModel
  attemptWithFallback = true  // Retry loop flag

  // Clear orphan messages
  for (const msg of assistantMessages) {
    yield { type: 'tombstone', message: msg }  // Remove from UI/Transcript
  }

  // Discard pending tool results with old tool_use_id
  streamingToolExecutor.discard()
  streamingToolExecutor = new StreamingToolExecutor(...)

  // Thinking signatures are model-bound; degraded model lacks capybara signature
permissions
  if (process.env.USER_TYPE === 'ant') {
    messagesForQuery = stripSignatureBlocks(messagesForQuery)
  }

  yield createSystemMessage(`Switched to ${renderModelName(innerError.fallbackModel)} ...`,
'warning')
```

```
  continue
}
```

IV. Streaming Fault Tolerance

4.1 Streaming Tool Executor

When the `streamingToolExecution` feature is enabled, tools can be **executed in parallel** while the model is streaming output, rather than waiting for the complete response:

```
const streamingToolExecutor = new StreamingToolExecutor(
  toolUseContext.options.tools,
  canUseTool,
  toolUseContext,
)

// Register tools in real-time within the streaming loop
for (const toolBlock of msgToolUseBlocks) {
  streamingToolExecutor.addTool(toolBlock, message)
}

// Real-time retrieval of completed tool results
for (const result of streamingToolExecutor.getCompletedResults()) {
  yield result.message
  toolResults.push(result.message)
}
```

> **Note**
>
> This means when the model streams multiple parallel tool calls, **as soon as one tool_use block is fully output, that tool begins executing immediately**, rather than waiting for all blocks to finish before executing them collectively.

4.2 Orphan Message Handling (Tombstoning)

When a streaming fallback occurs, some assistant messages (e.g., thinking blocks) have invalid signatures, and retrying directly would cause API errors:

```
if (streamingFallbackOccured) {
  for (const msg of assistantMessages) {
    yield { type: 'tombstone', message: msg }  // Remove from UI
  }
  logEvent('tengu_orphaned_messages_tombstoned', { orphanedMessageCount:
assistantMessages.length, ... })
  assistantMessages.length = 0
  streamingToolExecutor.discard()  // Discard results with old tool_use_id
  streamingToolExecutor = new StreamingToolExecutor(...)
}
```

> **Note**

- Streaming fallback: when an error occurs mid-streaming output, discard all received half-finished fragments, resend the request in non-streaming mode, and ultimately present the user with only one complete AssistantMessage
- Due to 4.1 Streaming Tool Executor above, results with the old tool_use_id must be discarded to prevent timing and execution confusion.

4.3 Tool Result Budget (Tool Result Size Limit)

```
messagesForQuery = await applyToolResultBudget(
  messagesForQuery,
  toolUseContext.contentReplacementState,
  persistReplacements ? records => recordContentReplacement(records, toolUseContext.agentId)
: undefined,
  new Set(toolUseContext.options.tools.filter(t =>
!Number.isFinite(t.maxResultSizeChars)).map(t => t.name)),
)
```

- Enforces **per-message budget** on aggregated tool result sizes
- Orthogonal to microcompact (both are non-interfering; microcompact operates only by `tool_use_id`)
- Replacement records are only persisted for `agent:*` and `repl_main_thread` query sources (supporting `/resume` and AgentTool recovery)

Orthogonality of `applyToolResultBudget` and microcompact: These two compression mechanisms operate on different dimensions of messages — `applyToolResultBudget` compresses tool result size **within a single message** (by character count limit), while microcompact compresses **relationships between messages** (by tool result count threshold and cache status).

Why only persist for specific querySource: Tool result replacement records need to be written to the filesystem (`recordContentReplacement` writes to sidechain or session file). For ephemeral subagent invocations (e.g., `agent_summary`), these records quickly become invalid (session state is entirely different after subagent execution completes), making disk writes pure waste. By limiting persistence to "main sessions" and "agents with `/resume` capability," the system avoids unnecessary disk I/O.

V. Max Output Tokens Recovery

A **three-level recovery** when the model output hits the `max_output_tokens` limit:

5.1 Level One: Token Limit Escalation

Only when the Statsig feature `tengu_otk_slot_v1` is enabled, if the default 8k limit is detected in use, escalate to `ESCALATED_MAX_TOKENS` (64k) and retry (within the same turn):

```
if (capEnabled && maxOutputTokensOverride === undefined
    && !process.env.CLAUDE_CODE_MAX_OUTPUT_TOKENS) {
  state = {
    ...state,
    maxOutputTokensOverride: ESCALATED_MAX_TOKENS,
    transition: { reason: 'max_output_tokens_escalate' },
  }
```

```
  continue
}
```

5.2 Level Two: Multi-Round Recovery (`MAX_OUTPUT_TOKENS_RECOVERY_LIMIT = 3`)

```
const recoveryMessage = createUserMessage({
  content: `Output token limit hit. Resume directly — no apology, no recap. ` +
           `Pick up mid-thought if that is where the cut happened. ` +
           `Break remaining work into smaller pieces.`,
  isMeta: true,
})
const next: State = {
  messages: [...messagesForQuery, ...assistantMessages, recoveryMessage],
  maxOutputTokensRecoveryCount: maxOutputTokensRecoveryCount + 1,
  ...
}
state = next; continue
```

> **Note**
>
> This is actually **dynamically inserting/intervening with a contextual prompt**
>
> - **A directive is constructed and injected** into the conversation history, then `continue` re-runs the API: `You were cut off. Don't apologize, don't recap. Resume directly where you left off. If not done, break remaining work into smaller chunks.`
> - This message is for the model, not the user (`isMeta: true` means it doesn't appear in the UI)

5.3 Level Three: Expose Error

If all three levels are exhausted, expose the withheld error message to the user:

```
if (isWithheldMaxOutputTokens(lastMessage)) {
  yield lastMessage  // Expose error
}
```

VI. Token Budget Mechanism

`query/tokenBudget.ts` implements an "early stop" mechanism to prevent continuing when the token budget is about to be exhausted:

```
type BudgetTracker = {
  continuationCount: number   // Number of auto-continuations
  lastDeltaTokens: number      // Token delta since last turn
  lastGlobalTurnTokens: number // Total token count from last turn
  startedAt: number
}

function checkTokenBudget(tracker, agentId, budget, globalTurnTokens): TokenBudgetDecision
```

Continuation conditions:

```
  /**
   * Diminishing Returns Detection
   * Three conditions must all be met:
   *   1. At least 3 consecutive checks have passed (this turn's conversation has run
several rounds)
   *   2. Tokens produced this round < 500 (this round's output is small)
   *   3. Tokens produced last round < 500 (last round's output was also small)
   *
   * Meaning: The model has been outputting only tiny amounts across multiple consecutive
rounds,
   * indicating the task is near completion.
   * Continuing to let the model run will only waste budget — stop this turn.
   */
  const isDiminishing =
    tracker.continuationCount >= 3 &&      // ① Consecutive check count threshold met
    deltaSinceLastCheck < DIMINISHING_THRESHOLD &&  // ② This round's delta < 500 tokens
    tracker.lastDeltaTokens < DIMINISHING_THRESHOLD  // ③ Last round's delta < 500 tokens

  /**
   * Passes check: Not yet diminishing AND under 90% of budget
   * → Allow model to continue, give user a nudge showing current consumption progress.
   *
   * Otherwise, stop if either condition is met:
   *   - Already diminishing (model output getting shorter, near completion)
   *   - Already over 90% of budget (remaining space is running out)
   */
  if (!isDiminishing && turnTokens < budget * COMPLETION_THRESHOLD) {  // < 90%
    return { action: 'continue', nudgeMessage: '...' }
  }
```

Stop conditions:

- Token usage exceeds 90% of budget, or
- Token deltas across 3 consecutive rounds are all < 500 tokens (**diminishing returns detection**), or
- No budget (agentId subagent scenario)

Meta message during auto-continuation: Shows user progress percentage:

```
Token budget continuation #2: 67% (335,000 / 500,000)
```

6.1 Design Intent of Diminishing Returns Detection

The "diminishing returns detection" in `checkTokenBudget` (`continuationCount >= 3 && deltaSinceLastCheck < 500`) is an **adaptive stopping mechanism** that acknowledges a reality: an Agent's work output does not grow linearly with token consumption.

Specifically: the Agent solves most problems in the first few chunks ("easy wins"), and subsequent chunks mainly handle "edge cases" and "repeated verification." If token deltas across 3 consecutive rounds are all less than 500, the model is only adding a small amount of new content each round — which may mean:

1. The work is nearly complete (best case), or
2. The model is stuck in repetitive labor or going in circles (worst case)

The code chooses **conservative stopping** — better to terminate early than let the model consume the budget indefinitely. This is a pragmatic design philosophy: budget guardrails are more reliable than precise completion judgments in the face of unpredictable Agent behavior.

> **Note**
>
> - The elegant design here is using **token output rate to determine whether the model is in a working state or an idle state**
> - But this design is relatively brute-force → consider using dynamic context injection to intervene in LLM behavior

6.2 `lastGlobalTurnTokens` and Cross-Turn Accumulation Tracking

The design of `BudgetTracker.lastGlobalTurnTokens` reveals a question: **where is token budget consumption calculated?**

`getTurnOutputTokens()` comes from `bootstrap/state.ts` and records **total token consumption for the entire turn** (including all API rounds). But `deltaSinceLastCheck = globalTurnTokens - tracker.lastGlobalTurnTokens` calculates the **delta between two adjacent checkpoints**. This means:

- If a turn includes multiple API calls (multiple rounds of tool execution), all those tokens accumulate into `globalTurnTokens`
- But `lastDeltaTokens` only records "delta from the last budget checkpoint to now," not "total consumption of the last turn"

> **Note**
>
> Primarily for **instantaneous change detection** — `deltaSinceLastCheck` cares about "how much more was consumed this round compared to the last round," not "what is the total for this round." The latter is handled by the `budget * COMPLETION_THRESHOLD` absolute threshold.

VII. Stop Hooks Mechanism

`query/stopHooks.ts` executes after each tool call completes (`needsFollowUp=false`):

```
const stopHookResult = yield* handleStopHooks(
  messagesForQuery, assistantMessages, systemPrompt, userContext,
  systemContext, toolUseContext, querySource, stopHookActive,
)
```

stopHookResult structure:

```
  type StopHookResult = {
    blockingErrors: Message[]     // Error messages reported by hooks (can accumulate
multiple)
    preventContinuation: boolean // Hook explicitly requests termination
  }
```

7.1 Execution Flow

1. **Save CacheSafeParams**: Available for `/btw` and SDK `control_request`
2. **Template Job Classification**: Updates `state.json` after each turn
3. **Background Tasks** (fire-and-forget):
   - Prompt Suggestion
   - Memory Extraction (`EXTRACT_MEMORIES`)
   - Auto Dream
4. **Stop Hook Execution**: Core blocking logic

> **Note**
>
> `/btw` and SDK `control_request` need to initiate a new API request but also want to reuse cache from the main loop — Stop Hooks quietly snapshot the main loop's API request parameters at the end of each turn, making them directly available for these two scenarios and saving the overhead of recreating prompt cache.
>
> - `/btw` : While the main loop is working, the user doesn't interrupt it; instead, a side question pops up to get an answer
> - `control_request` : SDK host applications (e.g., IDE plugins, external schedulers) can remotely send a control_request to a running Claude Code session, producing the exact same effect as `/btw` : read the main loop's context, initiate a single-round fork query, return the answer

7.2 Blocking Handling

```
if (stopHookResult.blockingErrors.length > 0) {
  state = {
    messages: [...messagesForQuery, ...assistantMessages, ...stopHookResult.blockingErrors],
    stopHookActive: true,  // Prevents infinite recursion
    transition: { reason: 'stop_hook_blocking' },
  }
  continue  // Inject blocking error into next round; model sees it and self-corrects
}
```

Infinite Loop Protection: `hasAttemptedReactiveCompact` is kept from being reset. If reactive compact has already failed once (indicating the context is genuinely over limit), a stop hook's blocking error triggering compact again would enter a dead loop — this protection keeps `hasAttemptedReactiveCompact` as `true` , avoiding repeated compression attempts.

Deep Meaning of `hasAttemptedReactiveCompact` Persistence: The semantic meaning of this field is not "has been attempted," but "compression is ineffective for the current context." Once it is set to `true` , subsequent stop hook blocking errors cannot trigger new compression attempts — because it has been proven that compression cannot solve the problem, and continuing to try would only waste API calls and potentially expand the context.

This effectively creates a **correlated memory between stop hook blocking errors and reactive compact failures**: stop hooks believe the error can be solved by correcting model behavior, but if reactive compact has already shown the problem lies not in model behavior but in context size, stop hooks' intervention is ineffective.

> **Note**
>
> REACTIVE_COMPACT is disabled by default in the current version; it is still a feature under testing

7.3 Prevent Continuation

If a hook explicitly requests stopping:

```
if (stopHookResult.preventContinuation) {
  return { reason: 'stop_hook_prevented' }
}
```

7.4 Teammate Hook Chain

For teammates in team mode, **after all regular Stop Hooks have fully passed** (no blocking errors, no preventContinuation), the following are additionally executed:

- `TaskCompleted` hooks (for in_progress tasks owned by this teammate)
  - Query the task list to find **in-progress tasks this teammate is responsible for**
  - Execute the TaskCompleted hook once for each task
- `TeammateIdle` hooks
  - After all TaskCompleted hooks have completed, execute the TeammateIdle hook once. This hook is not associated with any specific task; it concerns whether this teammate as a whole is idle

---

VIII. User Interruption Handling

8.1 Streaming Interruption (User Ctrl+C)

Trigger condition: `toolUseContext.abortController.signal.aborted === true` (user presses ESC to reject tool / Ctrl+C / SDK initiates abort)
Handling principle: Give started tools a proper conclusion; leave no orphan `tool_use` blocks

```
  /**
   * Step One: Consume remaining results of started tools
   *
   * Background: During streaming output, once a tool_use block is fully output, tool
   * execution begins immediately. When the user interrupts, some tools may:
   *   - Still be in the queue (not started)
   *   - Currently executing (in-flight)
   *
   * getRemainingResults() iterates all submitted tools:
   *   - Queued, not started: skip directly (don't execute)
   *   - Currently executing: wait for completion, then generate synthetic tool_result:
   *       "<tool_use_error>Error: Interrupted by user</tool_use_error>"
   *
   * Key: If getRemainingResults() is not called, these tool_use blocks will never have
   * corresponding tool_result blocks, resulting in an incomplete message tree structure,
   * confusing the API in the next round.
   */
  if (toolUseContext.abortController.signal.aborted) {
    if (streamingToolExecutor) {
      // Actively pull remaining results (drives executor to complete in-flight tools and
generate synthetic errors)
      for await (const update of streamingToolExecutor.getRemainingResults()) {
        if (update.message) yield update.message
```

```
      }
    } else {
      // Non-streaming tool execution mode (no streamingToolExecutor):
      // Find all tool_use blocks, inject "Interrupted by user" tool_result for each
      yield* yieldMissingToolResultBlocks(
        assistantMessages,
        'Interrupted by user',
      )
    }

    /**
     * Step Two: Computer Use Cleanup (CHICAGO_MCP main thread only)
     *
     * In Computer Use mode:
     *   - Main thread holds a process-wide lock (isLockHeldLocally)
     *   - Main thread holds a hidden window (isHidden)
     *
     * On user interruption:
     *   - Release the lock (let other tasks continue)
     *   - Unhide (restore window visibility)
     *
     * Main thread only: subagent cleanup would incorrectly release the main thread's lock,
     * causing the main thread to subsequently see isLockHeldLocally()===false and stop
     * sending exit notifications.
     */
    if (feature('CHICAGO_MCP') && !toolUseContext.agentId) {
      try {
        const { cleanupComputerUseAfterTurn } = await import(
          './utils/computerUse/cleanup.js'
        )
        await cleanupComputerUseAfterTurn(toolUseContext)
      } catch {
        // Silent failure: this is dogfooding cleanup, not on the critical path
      }
    }

    /**
     * Step Three: Send User Interruption Message
     *
     * Note: submit-interrupt scenarios (user submits a new message while interrupting the
current turn)
     * don't need this message — because the user's next submitted message already provides
sufficient context,
     * sending an extra "Interrupted by user" creates information noise.
     * Determination: abort reason === 'interrupt' (triggered by submitInterrupt)
     */
    if (toolUseContext.abortController.signal.reason !== 'interrupt') {
      yield createUserInterruptionMessage({
        toolUse: false,  // Interrupted model thinking/output, not a turn with tools
      })
    }
```

```
    return { reason: 'aborted_streaming' }
  }
```

8.2 Tool Execution Interruption

```
if (toolUseContext.abortController.signal.aborted) {
  // ... same cleanup logic as 8.1
  if (toolUseContext.abortController.signal.reason !== 'interrupt') {
    yield createUserInterruptionMessage({ toolUse: true })
  }
  if (maxTurns && nextTurnCountOnAbort > maxTurns) {
    yield createAttachmentMessage({ type: 'max_turns_reached', ... })
  }
  return { reason: 'aborted_tools' }
}
```

8.3 Submit Interrupt

Skip sending the interruption message (`reason !== 'interrupt'`) because the queued user message that follows already provides sufficient context.

IX. Async Prefetch and Parallelization

9.1 Memory Prefetch

```
using pendingMemoryPrefetch = startRelevantMemoryPrefetch(state.messages, toolUseContext)
// `using` declaration ensures cleanup on all exit paths

// Within the API streaming loop: consume prefetch results (only when settled)
if (pendingMemoryPrefetch.settledAt !== null && pendingMemoryPrefetch.consumedOnIteration
=== -1) {
  const memoryAttachments = filterDuplicateMemoryAttachments(
    await pendingMemoryPrefetch.promise,
    toolUseContext.readFileState,
  )
  for (const memAttachment of memoryAttachments) {
    yield createAttachmentMessage(memAttachment)
    toolResults.push(msg)
  }
  pendingMemoryPrefetch.consumedOnIteration = turnCount - 1
}
```

Note

- Uses `using` (TC39 Resource Management) to ensure `pendingMemoryPrefetch` is cleaned up on all exit paths
  - Regardless of how the query loop exits, background memory searches are cancelled and logs are recorded
- `consumedOnIteration` prevents duplicate consumption within the same iteration cycle (yielding to the user) — note this field is compared **across iterations** (`turnCount - 1`), not within the same iteration

```
什么时候 prefetch 会被"消费"（yield 给用户）？

1. prefetch 的 Promise 已经有结果了（settledAt !== null）
2. 之前没有被消费过（consumedOnIteration === -1）

消费之后，立刻把当前轮次记录下来（consumedOnIteration = turnCount - 1）。

为什么要用 turnCount - 1？

因为 prefetch 只能在工具执行完毕之后才会被检查和消费。来看一轮 query 循环的内部顺序：

第 N 轮
├── 预检阶段（SNIP / Microcompact / Autocompact）
├── API 流式循环 ← prefetch 在这里 settle（Promise 有结果了）
├── 工具执行（StreamingToolExecutor / runTools）← 这里是唯一检查 prefetch 的地方
└── → continue → 进入第 N+1 轮

prefetch 在 API 流式循环中就 settle 了，但代码检查 prefetch
的位置在"工具执行之后"——工具执行本身发生在同一轮里，所以 prefetch 实际上是在第 N
轮的工具执行完毕处被消费的，不是等到第 N+1 轮。
```

- `readFileState` filters out file states already processed by the model — this is cross-iteration cumulative filtering, ensuring that even if prefetch results are consumed multiple iterations later, already-"known" file states are not re-injected into the model.

9.2 Skill Discovery Prefetch

```
const pendingSkillPrefetch = skillPrefetch?.startSkillDiscoveryPrefetch(
  null, messages, toolUseContext,
)
// Consumed asynchronously during tool execution
```

Note

- Note this section has no specific code implementation; it is inferred from the interface and comments. The following important technical details are unknown:
  - How startSkillDiscoveryPrefetch is internally implemented (what model, what prompt)
  - The specific judgment logic of findWritePivot
  - How collectSkillDiscoveryPrefetch computes the hidden_by_main_turn boolean value
  - What the source of skills searched is (filesystem? vector retrieval?)
- Skill Discovery Prefetch is a background async discovery + consumption mechanism alongside tool results: on turns where tools that may involve write operations are called, it uses findWritePivot to determine triggering, silently prepares potentially needed skill attachments while the model reasons and tools execute, finally injects them alongside tool results, and uses hidden_by_main_turn to ensure the user never perceives discovery latency — transforming what was originally a per-turn blocking cost into an almost free side effect

9.3 Tool Use Summary

```
nextPendingToolUseSummary = generateToolUseSummary({
  tools: toolInfoForSummary,
  signal: toolUseContext.abortController.signal,
  isNonInteractiveSession: toolUseContext.options.isNonInteractiveSession,
  lastAssistantText,
})
.then(summary => summary ? createToolUseSummaryMessage(summary, toolUseIds) : null)
.catch(() => null)
```

After each turn's tool execution completes, "which tools were executed, doing what" is packaged and sent to Haiku (small model), so Haiku generates a one-line human-readable summary (e.g., "Fixed NPE in AuthService"), displayed in the mobile UI:

- The result is yielded at the next turn's `pendingToolUseSummary`
- Subagents (`toolUseContext.agentId`) skip this step (not displayed in mobile UI)

> **Note**
>
> The model cannot see this summary (context unchanged); it is information exclusive to the user (displayed in UI)

X. Tool Use Context Refresh

```
if (updatedToolUseContext.options.refreshTools) {
  const refreshedTools = updatedToolUseContext.options.refreshTools()
  if (refreshedTools !== updatedToolUseContext.options.tools) {
    updatedToolUseContext = {
      ...updatedToolUseContext,
      options: { ...updatedToolUseContext.options, tools: refreshedTools },
    }
  }
}
```

- Called whenever tools are needed; always gets the latest state
- Ensures newly connected MCP servers become available across multiple turns

> **Note**
>
> **Why `refreshTools` is a function rather than a value**
>
> `toolUseContext.options.refreshTools` is a **function** (`() => Tools[]`), not a snapshot of the tool array. The motivation behind this design choice: **the tool list may change during turn execution**.
>
> Typical scenario: a user connects a new MCP server while tools are executing. MCP server connections are asynchronous events that can occur at any time. If `refreshTools` were a snapshot value rather than a function, the query loop would need to call it again on every possible exit path to ensure it sees the latest tool list — easy to miss and inelegant.
>
> By designing it as a function, the query loop only needs to **call on demand** (`refreshTools()`) whenever it needs the tool list, getting the current latest state each time — this is essentially **lazy evaluation** applied to tool management: the tool list is not determined at turn start, but fetched fresh each time it is needed.

XI. Error Classification and Handling

11.1 Error Type Mapping

Error	Handling	Exit Reason
`ImageSizeError` / `ImageResizeError`	Yield user-friendly message	`image_error`

Error	Handling	Exit Reason
Model runtime exception	Yield orphan tool_result + real error message	`model_error`
`FallbackTriggeredError`	Switch model, retry	Continue loop
`PROMPT_TOO_LONG` (pre-call)	Yield error message	`blocking_limit`
413 withheld → recovery failed	Expose withheld error	`prompt_too_long`
Media error recovery failed	Expose withheld error	`image_error`
Stop hook blocking	Inject error message, continue	Continue loop
Tool execution interrupted	Generate synthetic tool_result	`aborted_tools`
`hook_stopped_continuation`	Stop	`hook_stopped`
max_turns reached	Yield attachment message	`max_turns`
Stop hook prevented	Stop	`stop_hook_prevented`

11.2 Safety Guarantees for Error Exposure

```
// Prevent exposing misleading "[Request interrupted by user]"
// SDK consumers may see phantom interrupt and ignore the real cause
yield createAssistantAPIErrorMessage({ content: errorMessage })
logAntError('Query error', error)  // Loud logging in ants mode
return { reason: 'model_error', error }
```

Note

- This is an error attribution transparency guarantee: **exposing phantom interrupts that look like user cancellations but are actually SDK bugs as real errors prevents users from misjudging, thereby helping real bugs be discovered and fixed**
- In reality, the Node.js runtime may throw an exception (e.g., a method doesn't exist), but the SDK's outermost layer doesn't handle it finely, wrapping it uniformly as "Request interrupted by user" → queryModelWithStreaming very rarely throws exceptions directly (rather than normally yielding errors)

XII. Feature Flags Summary

Feature Name	Purpose	Related File
`REACTIVE_COMPACT`	Enable reactive compression	`services/compact/reactiveCompact.js`
`CONTEXT_COLLAPSE`	Enable context collapse	`services/contextCollapse/index.js`
`HISTORY_SNIP`	Enable historical message trimming	`services/compact/snipCompact.js`
`CACHED_MICROCOMPACT`	Enable cached microcompression	`compact.ts`
`TOKEN_BUDGET`	Enable token budget early stopping	`query/tokenBudget.ts`
`UNATTENDED_RETRY`	Enable persistent retry	`services/api/withRetry.ts`
`BG_SESSIONS`	Enable background task summarization	`utils/taskSummary.js`
`CHICAGO_MCP`	Enable MCP cleanup hooks	`utils/computerUse/cleanup.js`
`EXTRACT_MEMORIES`	Enable memory extraction	`services/extractMemories/`

Feature Name	Purpose	Related File
`EXPERIMENTAL_SKILL_SEARCH`	Enable skill pre-search	`services/skillSearch/prefetch.js`
`TEMPLATES`	Enable template task classification	`jobs/classifier.js`

All features are accessed via `feature('NAME')`, and `bun:bundle`'s tree-shaking ensures disabled feature strings don't enter external builds.

12.1 `querySource` as a Typed Security Policy Context

`querySource` is an **imperative label** that answers "who is calling query()" — not just a string identifier. It serves as the basis for policy dispatch:

- `"compact"` / `"session_memory"`: Subagents, skip blocking limit check (deadlock prevention)
- `"repl_main_thread"` / `"sdk"`: Main session, save CacheSafeParams
- `"agent:*"`: Subagents, persist tool result replacement records
- Background sources: Fail immediately on 529, no retry

The key insight of this design: **the caller knows its own semantics better than the query loop itself**. The query loop doesn't need to understand "what a compact agent is," only that "compact-source calls need to be exempted from blocking checks" — this policy knowledge is passed from the caller to query, not derived by query itself.

XIII. Design Philosophy Summary

> Before reading the following sections, review the **"Three Core Design Principles"** from Section I: state machine pattern, unified error recovery exit, and cooperative concurrency. These principles permeate the entire query.ts and are the foundation for understanding the motivation behind every specific mechanism.

13.0 The "Three-Layer Funnel" Philosophy of Error Handling

query.ts's error handling follows a **unified "three-layer funnel" philosophy**, rather than writing separate branches for each error:

- **Layer One (Withhold)**: Capture recoverable errors in the streaming loop, but don't expose them to the caller. Reason: SDK consumers may terminate the session upon seeing an `error` field, preventing subsequent recovery logic from ever executing.
- **Layer Two (Recover)**: Attempt automatic recovery after the turn ends: PTL goes collapse drain → reactive compact → ultimately expose; max_output_tokens goes 8k→64k escalation → multi-round recovery message → ultimately expose; 529 goes exponential backoff → model fallback → ultimately expose.
- **Layer Three (Expose)**: After all recovery paths are exhausted, expose the error and exit. At this point `executeStopFailureHooks` is called (rather than the full stop hooks chain), preventing the death spiral caused by stop hooks injecting more tokens when the model never produced a valid response.

13.1 Defensive Streaming Handling

Unlike the traditional "call the API completely first, then process results" model, query.ts **in real time** processes tools, errors, and interruptions during streaming output. The advantages of this design:

- **Parallelization**: Tools begin executing while the model is still outputting
- **Low latency**: Errors are caught immediately upon generation, rather than waiting for the full response

- **Resource safety**: Long-running tasks can be interrupted instantly

13.2 Withheld Mode

For recoverable errors (413 PTL, max_output_tokens, media size), choosing to **withhold** rather than immediately expose is because SDK consumers terminate the session upon seeing an `error` field, preventing recovery logic from ever executing. The withheld mechanism ensures recovery paths have sufficient opportunity to complete.

13.3 Immutable State Transitions

All loop state updates are done by **replacing the entire State object**:

```
state = { ...state, newField: newValue }
```

Rather than modifying fields one by one, which makes:

- State structure consistent at all continue jump points
- Easy to verify recovery paths in tests via the `state.transition` field
- Avoids "certain fields not correctly reset in certain branches" bugs

13.4 Multi-Layer Degradation Strategy

Each error type is designed with **at least two layers** of degradation paths:

- Prompt Too Long: `pre-call block` → `context collapse drain` → `reactive compact` → `expose error`
- Max Output Tokens: `8k→64k escalation` → `multi-round recovery message` → `expose error`
- 529 Overload: `retry×3` → `model fallback` → `expose error`

13.5 Dependency Injection: Intentional Design for Testability

The `deps.ts` in query.ts implements a dependency injection (DI) pattern, specifically addressing testing challenges: at the time, 6-8 test files each used `spyOn` to intercept functions from different modules, causing import + spy boilerplate to proliferate.

```
// Design principles of deps.ts:
// 1. Types derived from real implementations (typeof fn), ensuring signatures stay in sync
// 2. Production uses productionDeps() to get real implementations
// 3. Tests inject fake implementations via QueryParams.deps
// 4. Scope deliberately kept narrow (only 4 dependencies), proving the pattern before
expanding
```

This is a "minimum viable DI": rather than introducing an IOC container across the entire system, a thin injection layer is provided in the module that most needs testability (query.ts). The `deps` object itself is a plain TypeScript object (not a class/container), requiring no framework support for injection.

13.6 529 Overload "Amplification Attack" Protection

A subtle global design consideration in the 529 error handling logic: when the API Gateway is in a capacity overload (529) state, **every retry request actually increases the gateway's burden**, forming a positive feedback loop — unlike ordinary 429 rate limiting, which is local quota exhaustion where retries don't affect others.

The `FOREGROUND_529_RETRY_SOURCES` design directly addresses this: only query sources where the user is waiting in the foreground retry on 529; all background tasks (summary generation, title generation, classifiers, etc.) encountering 529 **fail immediately**, never retry. This minimizes the request amplification factor during 529 — because background task failures don't affect user experience, while foreground user requests can briefly wait and retry.

Also note that when 529 triggers model fallback, only **non-subscribing users** (`!isClaudeAISubscriber()`) or non-custom Opus models fall back. Paying users don't go through the fallback path — their enterprise agreements typically include PAYG (pay-as-you-go), making retry more reasonable than fallback.

13.7 Background Task "Avalanche Prevention" Isolation

Another design point related to 529 overload protection: background tasks (`session_memory` queries, `compact` agents, etc.) encountering context overflow would trigger deadlock — because these subprocesses themselves need to run in order to **reduce** token usage, if they're blocked at invocation time due to too many tokens, they'll never be able to execute.

This isolation principle is reflected throughout the code:

- Blocking limit checks skip `compact` and `session_memory` query sources
- Microcompact skips `compact` and `session_memory` query sources (prevents subagent and parent agent both trying to compress and waiting for each other)

This is essentially **resource-ordering deadlock prevention**: if A needs B to complete first before it can run, and B also needs A to complete first before it can run, the system deadlocks. By exempting "resource-consuming subtasks" from resource-based blocking, this classic deadlock pattern is avoided.

13.8 QueryCheckpoint Dual-Layer Performance Tracking System

The `queryCheckpoint()` calls scattered throughout query.ts actually belong to a **dual-layer performance tracking system**, targeting different use cases:

Layer One: Headless Profiler (`headlessProfilerCheckpoint`) tracks **end-to-end latency breakdown from when the user presses Enter to when the first character appears**, in headless mode. This data is sent to a centralized monitoring system for oncall engineers to decide whether to scale or roll back.

Layer Two: Query Profiler (`queryCheckpoint` + `utils/queryProfiler.ts`) tracks **stage-by-stage timing within a single query**, answering "where is it slow this time?" This data is enabled via the `CLAUDE_CODE_PROFILE_QUERY=1` environment variable and output to debug logs during local development.

The layered design of both avoids "using one tool to satisfy both monitoring and debugging needs simultaneously, resulting in neither being done well." Monitoring requires low overhead, good aggregation, and no interference with production; debugging requires granularity, immediate feedback, and local analyzability.

13.9 "Implicit Dependency" Awareness in Tool Execution

The design of StreamingToolExecutor where Bash errors trigger cascading cancellation (sibling cancellation) reveals a deep awareness of **implicit dependencies**:

- Multiple Bash commands typically have implicit file/directory dependencies between them. The cascading cancellation mechanism of the `hasErrored` flag is an encoding of these implicit dependencies: by letting the first Bash error "kill" all executing sibling tools, the system acknowledges the dependency relationship between Bash commands and stops the bleeding immediately.

- But this doesn't mean all tools follow the same rule — read-operation tools (Read/WebFetch/Grep/Glob) are **concurrently safe**; there are no implicit dependencies between them, so they can run in parallel. This is a refined concurrency control strategy: conservative (serial) for operations whose dependency is uncertain, aggressive (parallel) for operations confirmed to have no dependencies.

13.10 Analytics as a First-Class Citizen of Architecture

Scattered throughout query.ts are numerous `logEvent()` calls — not "post-hoc instrumentation added in late development," but part of the system architecture.

Analytics events are designed alongside the feature at the same time, not retrofitted after launch. Every key business decision point (autocompact trigger, model fallback, token budget stop, streaming tool execution) synchronously outputs structured data fields. These fields are designed from the start to answer specific product questions (e.g., "how much did compression actually save in costs?", "was token budget stopped proactively or passively?"), enabling product teams to make decisions based on data rather than guesswork or log inspection. Product logic and analytics logic are the same code, the same PR, the same review — field naming is consistent, data formats align, validated before launch, historical data is traceable. This turns "data-driven decision-making" from a slogan into a system hard-coded into the architecture.

13.11 `using` Declarations and Deterministic Resource Management

Memory Prefetch uses the TC39 `using` declaration (resource management) rather than manual cleanup — this is a guarantee of **deterministic resource release**:

```
using pendingMemoryPrefetch = startRelevantMemoryPrefetch(state.messages, toolUseContext)
```

`using` is JavaScript's deterministic destruction mechanism (similar to Rust's `Drop` or Python's `with`). Whether the code exits the query loop via `return`, `throw`, or `continue`, the cleanup logic for `pendingMemoryPrefetch` is always called. This eliminates an entire class of "resource leak" bugs — developers don't need to remember to manually call cleanup on every exit path.

In a complex loop like query.ts with 9 different exit paths, deterministic cleanup is especially critical. Any single exit path missing cleanup accumulates leaks over long sessions.

13.12 Death Spiral Protection

Multiple places in the code explicitly avoid falling through to stop hooks when **the model has not produced a valid response**:

```
// Do NOT fall through to stop hooks: the model never produced a valid response,
// so hooks have nothing meaningful to evaluate. Running stop hooks
// on prompt-too-long creates a death spiral:
// error → hook blocking → retry → error → …
yield lastMessage
void executeStopFailureHooks(lastMessage, toolUseContext)
return { reason: isWithheldMedia ? 'image_error' : 'prompt_too_long' }
```

Similarly, the persistence design of `hasAttemptedReactiveCompact` prevents stop hook blocking errors from triggering compact into an infinite loop.

XIV. Key Data Flow Diagram

```
User Input
  │
  ▼
┌──────────────────────────────────────────────────────────────┐
│  queryLoop: while(true)                                      │
│  ┌──────────────────────────────────────────────────────┐    │
│  │ Pre-check Phase:                                      │  │
│  │  1. Memory Prefetch (async, non-blocking)            │  │
│  │  2. Skill Discovery Prefetch (async)                 │  │
│  │  3. HISTORY_SNIP ──→ Free token budget               │  │
│  │  4. Microcompact ──→ Cache optimization              │  │
│  │  5. Context Collapse ──→ Collapse                    │  │
│  │  6. Autocompact ──→ Summarization compression        │  │
│  │  7. [pre-call] Blocking limit check                  │  │
│  └──────────────────────────────────────────────────────┘    │
│                          │
│                          ▼
│  ┌──────────────────────────────────────────────────────┐    │
│  │ API Call Phase (withRetry wrapper)                   │  │
│  │  for await (message of deps.callModel())             │  │
│  │   ├─ Streaming tool registration → StreamingToolExecutor │
│  │   ├─ Real-time tool execution → getCompletedResults() │
│  │   ├─ Withheld check (PTL/media/MOT)                  │  │
│  │   └─ Orphan message Tombstoning (on fallback)        │  │
│  │                                                     │  │
│  │  Error handling:                                     │  │
│  │   ├─ 529 → Exponential backoff → FallbackTriggeredError │
│  │   ├─ 429/401/408/409/5xx → Exponential backoff retry   │
│  │   └─ max_tokens overflow → Dynamically lower max_tokens and retry │
│  └──────────────────────────────────────────────────────┘    │
│                          │
│                          ▼
│  ┌──────────────────────────────────────────────────────┐    │
│  │ Post-processing Phase:                               │  │
│  │   ├─ User interruption check → Synthetic tool_result │  │
│  │   ├─ Post-sampling Hooks (fire-and-forget)           │  │
│  │   ├─ needsFollowUp=false?                            │  │
│  │   │   ├─ PTL recovery: collapse drain → reactive compact │
│  │   │   ├─ MaxOutputTokens: escalate→multi-round→expose   │  │
│  │   │   ├─ Stop Hooks → blockingErrors → continue      │  │
│  │   │   ├─ Token Budget → continue/stop                │  │
│  │   │   └─ return { reason: 'completed' }              │  │
│  │   └─ needsFollowUp=true?                             │  │
│  │       ├─ Tool execution (Streaming / Blocking)       │  │
│  │       ├─ Memory Prefetch consumption                 │  │
│  │       ├─ Skill Discovery consumption                 │  │
│  │       ├─ MCP tool refresh                            │  │
│  │       └─ continue (next round)                       │  │
│  └──────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────┘
```

XV. Related Files Quick Reference

File	Responsibility
`restored-src/src/query.ts`	Main loop, context management, streaming fault tolerance, recovery path orchestration
`restored-src/src/query/stopHooks.ts`	Stop/TaskCompleted/TeammateIdle hooks execution
`restored-src/src/query/tokenBudget.ts`	Token budget tracking and early stop decisions
`restored-src/src/query/deps.ts`	Dependency injection (`deps.microcompact` / `deps.autocompact` / `deps.callModel`)
`restored-src/src/query/config.ts`	Immutable config snapshot (except Statsig gates)
`restored-src/src/services/api/withRetry.ts`	API retry wrapper, backoff strategy, model fallback
`restored-src/src/services/compact/reactiveCompact.ts`	Reactive compression (413/media error recovery)
`restored-src/src/services/compact/autoCompact.ts`	Proactive compression (turn-threshold triggered)
`restored-src/src/services/compact/compact.ts`	`buildPostCompactMessages` compression result construction
`restored-src/src/services/compact/snipCompact.ts`	Historical message trimming
`restored-src/src/services/compact/microcompact.ts`	Microcompression / Cached Microcompact
`restored-src/src/services/contextCollapse/index.ts`	Context collapse
`restored-src/src/services/tools/StreamingToolExecutor.ts`	Streaming tool parallel executor
`restored-src/src/services/tools/toolOrchestration.ts`	`runTools` blocking tool execution
`restored-src/src/utils/hooks.js` / `postSamplingHooks.ts`	Hook system
`restored-src/src/utils/tokenBudget.js`	`getBudgetContinuationMessage` progress messages
`restored-src/src/bootstrap/state.ts`	Global token budget state